Studio Nx

Backend(FastAPI/Python) 51 — check_rate_limit() · record_inquiry_send()

2026-10-08

Backend(FastAPI/Python) 51 — `check_rate_limit()` · `record_inquiry_send()`

항목	내용
파일	`studionx/backend/app/services/inquiry_service.py`
작업	P1-12 문의 API — 반복 전송 제한 · PRD 01 FR-113
작성	Claude (AI) · 2026-10-08 · 사용자 검토
이전 · 다음	50 `InquiryCreateRequest` · 52 `create_inquiry()`

1. 하는 일

같은 사람이 문의를 짧은 시간에 계속 보내지 못하게 막는다. **같은 이메일 또는 같은 IP 로 10분 안에 2번** 저장되면, 3번째는 429.

상황 (같은 이메일 · 같은 IP)	결과
1번째 · 2번째	통과
3번째 (10분 안)	429 `TOO_MANY_REQUESTS` "Too many messages. Please try again later." · `details.retry_after` = 몇 초 뒤 가능한지
IP 는 다르지만 이메일이 같음 (`A@Example.com` 도 같은 이메일)	막힘
이메일은 다르지만 IP 가 같음	막힘
다른 IP · 다른 이메일	통과
저장에 **실패한** 시도 · 막힌 시도	**세지 않음**
첫 번째로부터 10분이 지남	한 자리가 비어 다시 1번 가능

2. 언제 쓰나

52 `create_inquiry` 안에서:

순서	함수
① 저장 전	`check_rate_limit(ip, email)` — 이미 2번이면 여기서 429 (저장하지 않음)
② 저장(commit) 성공 뒤	`record_inquiry_send(ip, email)` — 이번 것을 셈

사용자 결정 (2026-10-08)

정한 것	결정	템플릿
횟수 · 시간	**10분에 2번**	10분에 3번
저장에 실패한 시도도 세나	**세지 않음**	(정하기)

템플릿은 `check_rate_limit` 하나가 "검사하고 통과하면 바로 셈" 이었다. 그러면 검사를 통과한 뒤 **저장이 실패해도** 이미 세어진다. 결정대로 하려면 세는 일을 저장 **뒤로** 옮겨야 해서, 함수를 둘로 나눴다.

메모리 방식은 2026-10-03 결정 그대로 — 서버를 다시 켜면 기록이 비고, 서버가 여러 대면 따로 센다 (Phase 1 은 서버 한 대).

3. 코드

import (파일 맨 위)

```
import threading
import time
from collections import deque

from sqlalchemy.orm import Session

from app.core.errors import AppError, ErrorCode
from app.models import Inquiry, User
from app.schemas.inquiry import InquiryCreateRequest
```

`threading` · `time` · `AppError` · `ErrorCode` 를 이번에 추가했다.

상수 · 기록 장소

```
INQUIRY_LIMIT_COUNT = 2  # 이 횟수까지 (2026-10-08 사용자 결정: 10분에 2번 — 템플릿은 3)
INQUIRY_LIMIT_SECONDS = 10 * 60  # 이 시간(초) 안에

_recent_sends: dict[str, deque[float]] = {}
_lock = threading.Lock()  # FastAPI 동기 함수는 여러 스레드에서 돎 → 기록을 함께 고치지 않게
_clock = time.monotonic  # 시스템 시계를 바꿔도 영향 없는 시각 (시험에서 바꿔 끼움)


def _keys(ip: str | None, email: str) -> list[str]:
    """제한을 셀 열쇠 — 이메일(소문자) 하나 + IP 가 있으면 IP 하나."""
    keys = [f"email:{email.strip().lower()}"]
    if ip:
        keys.append(f"ip:{ip}")
    return keys
```

함수

```
def check_rate_limit(ip: str | None, email: str) -> None:
    now = _clock()
    with _lock:
        for key in _keys(ip, email):
            q = _recent_sends.get(key)
            if q is None:
                continue
            while q and now - q[0] > INQUIRY_LIMIT_SECONDS:
                q.popleft()
            if not q:
                del _recent_sends[key]
                continue
            if len(q) >= INQUIRY_LIMIT_COUNT:
                raise AppError(
                    ErrorCode.TOO_MANY_REQUESTS,
                    429,
                    "Too many messages. Please try again later.",
                    details={"retry_after": int(INQUIRY_LIMIT_SECONDS - (now - q[0])) + 1},
                )
```

```
def record_inquiry_send(ip: str | None, email: str) -> None:
    now = _clock()
    with _lock:
        for key in _keys(ip, email):
            _recent_sends.setdefault(key, deque()).append(now)
```

문법 풀이

import 문법	뜻
`import threading`	여러 스레드가 같은 자료를 동시에 고치지 못하게 하는 잠금(`Lock`)
`import time`	`time.monotonic()` — 컴퓨터가 켜진 뒤 흐른 초. 시계를 바꿔도 뒤로 가지 않음
`from collections import deque`	양쪽 끝에 넣고 빼기가 빠른 목록 — 오래된 것은 앞에서 빼고(`popleft`) 새것은 뒤에 넣음(`append`)

식	뜻
`dict[str, deque[float]]`	열쇠(`"email:a@b.com"`)마다 저장 시각 목록
`with _lock:`	이 블록은 한 번에 한 스레드만 — 끝나면 저절로 풀림
`_clock = time.monotonic`	함수를 **변수에 담아 둠** — 시험에서 손으로 움직이는 시계로 바꿔 끼울 수 있게
`while q and now - q[0] > …: q.popleft()`	가장 오래된 기록부터 10분 넘은 것을 버림
`del _recent_sends[key]`	빈 목록은 열쇠째 지움 — 메모리가 계속 늘지 않게
`_recent_sends.setdefault(key, deque())`	없으면 빈 목록을 만들어 넣고, 있으면 그것을 돌려줌
`int(…) + 1`	남은 초를 올림 — 0초 남았다고 알려 주지 않게

4. 흐름

검사: 지금 시각 → 잠금 → 열쇠(이메일 · IP)마다 10분 넘은 기록 버리기 → 비면 지우기 → 2개 이상이면 429. **세기 (저장 성공 뒤):** 지금 시각 → 잠금 → 열쇠마다 시각 하나 추가.

5. 왜 이렇게

선택	이유
둘로 나눔 (검사 · 세기)	사용자 결정 "저장에 실패한 시도는 세지 않음" — 세는 시점을 저장 뒤로
이메일 **또는** IP	이메일만 보면 주소만 바꿔 계속 보낼 수 있고, IP 만 보면 같은 사람이 와이파이를 바꿔 보낼 수 있다. 둘 다 본다
이메일은 소문자 · 공백 제거로 비교	`A@Example.com` 과 `a@example.com` 은 같은 사람
IP 가 없으면 이메일만	시험 도구 등 IP 를 모르는 요청 (`ip:None` 이라는 열쇠를 만들지 않음 — 모두가 한 열쇠로 묶이는 것을 막음)
`time.monotonic`	서버 시계가 바뀌어도(시간 맞춤 등) 10분 계산이 틀어지지 않는다
잠금	FastAPI 의 동기 라우터는 스레드 여러 개에서 돈다. 동시에 같은 목록을 고치면 기록을 잃을 수 있다 (시험: 100개 스레드)

알아 둘 점 — 동시에 두 번: 같은 사람이 **같은 순간에** 두 번 보내면, 둘 다 검사를 통과한 뒤 저장되어 한도보다 1번 더 저장될 수 있다. 검사부터 저장까지를 한 잠금으로 묶으면 막을 수 있지만, 그동안 다른 사람의 문의까지

기다리게 된다. 문의 폼은 보내는 동안 버튼을 잠그므로(화면) 실제로는 드물다.

6. 시험 (`tests/test_inquiry_service.py` 새 파일 · 12개, 시계를 손으로 움직임)

시험	확인한 것
상수	2번 · 600초 (사용자 결정)
3번째	429 · 문구 · `retry_after` 601초
이메일 같고 IP 다름 (대소문자 · 공백도 다름) · IP 같고 이메일 다름	막힘
다른 사람	통과
IP 없음	이메일로만 셈, `ip:None` 열쇠 없음
검사만 5번 (저장 실패) 뒤 진짜 저장 2번	2번 모두 됨 → 3번째 막힘 — **실패는 세지 않음**
막힌 뒤 10분 지남	다시 됨 (막힌 것도 세지 않음)
`retry_after`	첫 기록 기준으로 줄어듦
10분 창이 밀림	첫 기록이 10분을 넘으면 1자리 남
오래된 기록	지워져 기록 장소가 빔
스레드 100개가 동시에 세기	하나도 잃지 않음

전체 시험 380개 통과, `ruff` 검사 통과.